BIG DATA · Grado en Ciencia e Ingeniería de Datos · ULPGC · 2026–2027

Assignment 2: Optimised Dense Matrix Multiplication and Sparse Matrices

How much do loop order, layout and blocking speed up dense multiplication, and when does a sparse representation pay off?

Amado Artiles Rodríguez

October 2026

Repository: https://github.com/amadoartiiles/matrix-multiplication-optimised-sparse

Abstract

Taking the validated C i-j-k kernel of Assignment 1 as the baseline, this study measures three sequential dense optimisations (i-k-j loop reordering, a transposed copy of B and 64×64 cache blocking) and an own CSR sparse multiplication (Gustavson's row-wise algorithm) on one laptop (AMD Ryzen 7 7730U, Windows 11, GCC 16 `-O2`), with five measured repetitions per configuration and fixed budgets of 300 s and 4 GB. All dense kernels returned bitwise-identical results, and all 97 validated configurations were within a rigorous error bound. Blocking was the fastest dense method, kept a constant cost of about 0.2 ns per multiply-add up to $n = 4096$ and was 27.8× faster than the baseline at $n = 2048$ and 31.8× at $n = 3072$; the baseline exceeded the time budget at $n = 4096$. Loop reordering matched blocking until B outgrew the 16 MB L3 cache and then became twice as slow; SIMD vectorisation contributed a factor of 1.25–1.7, the access pattern the rest. On uniformly random inputs, CSR beat the blocked kernel by more than two orders of magnitude at densities up to 1 %, by 1.6–2× at 25 % and lost at 50 %; the separately timed conversions dominated the CSR time up to 1–2 % density but never removed its advantage. CSR reached $n = 32768$ at 0.1 % density in 0.3 s, where dense storage alone would need 26 GB. The advantage depended on the output fill-in and the nonzero structure, not on input density alone: banded inputs with the same density and the same number of scalar products ran 2–5.6× faster than uniform ones. The conclusions apply to these implementations, inputs and machine only.

Keywords: matrix multiplication, loop reordering, cache blocking, vectorisation, sparse matrices, CSR, SpGEMM, benchmarking.

1 Introduction and objective

Assignment 1 showed that the textbook i-j-k triple loop in C is limited by memory access, not by arithmetic: its cost per multiply-add rose from 0.6 ns to almost 6 ns as the matrices left the caches, and a 16-byte row padding alone changed its runtime by up to 6.9×. This study takes that validated C kernel as the baseline and asks two separate questions.

Research questions

Part A. How do three sequential optimisations of dense multiplication (loop reordering, a transposed layout of B and cache blocking) change runtime and memory relative to the Assignment 1 baseline as the matrix size n grows, on one fixed machine?

Part B. For matched logical inputs, at which tested densities and nonzero structures does a CSR sparse multiplication run faster or use less memory than a dense method, and which costs (indexing, conversion, output fill-in) reduce its advantage?

Hypotheses (stated before measuring)

H1 The optimised dense kernels perform the same n^3 multiply-adds but read B with unit stride, so their cost per multiply-add stays nearly constant with n; their advantage over the baseline grows with n and is largest where the baseline suffers conflict misses ($n = 512, 1024, 2048$).

H2 Cache blocking is the fastest dense method at large n, because it also keeps a tile of B and of C in cache; loop reordering alone is close to it while a row of B fits in L2.

H3 The CSR product does work proportional to the number of scalar products, $\approx n^3 d^2$ for uniformly random inputs of density d, so it beats dense multiplication by a large factor at low density, and the advantage disappears at a density of the order of ten percent because each sparse product is several times more expensive than a dense one.

H4 Input density alone does not determine the result: the output can be much denser than the inputs, and inputs with the same density but different structure produce different output sizes and runtimes.

2 Background

Loop order and locality

All loop orders of $C_{ij} = \sum_k A_{ik} B_{kj}$ perform the same n^3 multiply-adds [1]. In the i-j-k order the innermost loop walks a *column* of the row-major matrix B, touching a new cache line on every iteration and, for power-of-two n, mapping all of them to a few cache sets (Assignment 1). In the i-k-j order the innermost loop is $C_{i,:} \mathrel{+}= A_{ik} B_{k,:}$: both B and C are read with unit stride, and the iterations are independent, so the compiler can vectorise them [2]. Transposing B first is the alternative: the dot product $\sum_k A_{ik} B^T_{jk}$ reads two rows, at the price of an n^2 copy and extra memory.

Cache blocking

Blocking (tiling) partitions the three loops into $b \times b$ tiles so that the working set of the inner loops (a tile of B and parts of the rows of C) stays in cache while it is reused; it reduces memory traffic from $O(n^3)$ to $O(n^3/b)$ words [3, 4]. The best b depends on the cache sizes and must be chosen empirically. Algorithmic alternatives such as Strassen's $O(n^{2.81})$ method [5] change the arithmetic and its rounding errors and were not used here, so that all dense methods remain directly comparable.

Vectorisation

Since GCC 12, `-O2` enables loop vectorisation with a "very cheap" cost model [6]. Without `-march`, x86-64 code uses 16-byte SSE2 vectors (two doubles). A loop of independent updates such as $C_{ij} \mathrel{+}= a B_{kj}$ can be vectorised without changing any result; a reduction such as $s \mathrel{+}= A_{ik} B_{kj}$ cannot, because reordering the sum would change rounding, which IEEE semantics without `-ffast-math` forbid.

Sparse storage and multiplication

Compressed Sparse Row (CSR) stores a matrix with z nonzeros as a row-pointer array of $n + 1$ entries, and column indices and values of length z [7]. Gustavson's row-wise algorithm [8, 9] computes row i of C as a combination of the rows of B selected by the nonzeros of row i of A, using a dense accumulator and a marker array of length n. Its work is the number of scalar products $W = \sum_k \operatorname{nnz}(A_{:,k}) \operatorname{nnz}(B_{k,:})$ plus $O(n)$ per row, independent of n^3 [10]. For independent uniformly random inputs of density d, $\mathbb{E}[W] = n^3 d^2$ and an entry of C is nonzero with probability $1 - (1 - d^2)^n$,

so the output fills in quickly: for $n = 2048$ it is already about 50 % dense at $d = 1.8\,\%$.

Error bound

As in Assignment 1, a computed dot product of length n satisfies $|\hat{C} - C| \leq \gamma_n |A||B|$ element-wise, with $\gamma_n = nu/(1 - nu)$ and $u = 2^{-53}$ [11, Sec. 3.5].

3 Methodology

3.1 Implementations

All methods are written in C11 by the author and compiled with the same flags; none calls a library. Figure 1 shows the dense kernels and Figure 2 the CSR kernel.

Figure 1: The four dense kernels (abridged from `c/dense.c`). Each allocates and zero-initialises C with `calloc` inside the timed region.

ijk (baseline) · c/dense.c, body copied unchanged from Assignment 1

```c
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) {
        double s = 0.0;
        for (int k = 0; k < n; k++) s += A[(size_t)i*n + k] * B[(size_t)k*n + j];
        C[(size_t)i*n + j] = s;
    }
```

ikj (loop reordering)

```c
for (int i = 0; i < n; i++) {
    double *Ci = C + (size_t)i*n;
    for (int k = 0; k < n; k++) {
        const double a = A[(size_t)i*n + k];
        const double *Bk = B + (size_t)k*n;
        for (int j = 0; j < n; j++) Ci[j] += a * Bk[j];      /* vectorised by GCC */
    }
}
```

transposed (data-layout change; the copy is inside the timed region)

```c
/* Bt = B^T, copied in 32x32 tiles */
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) {
        double s = 0.0;
        for (int k = 0; k < n; k++) s += Ai[k] * Bt[(size_t)j*n + k];
        C[(size_t)i*n + j] = s;
    }
```

blocked (cache blocking of the ikj nest, tile b = 64)

```
for (int ii = 0; ii < n; ii += b)
  for (int kk = 0; kk < n; kk += b)
    for (int jj = 0; jj < n; jj += b)
      for (int i = ii; i < min(ii+b, n); i++)
        for (int k = kk; k < min(kk+b, n); k++) {
          const double a = A[(size_t)i*n + k];
          for (int j = jj; j < min(jj+b, n); j++)
            C[(size_t)i*n + j] += a * B[(size_t)k*n + j];
        }
```

Identical arithmetic

In every dense kernel each C_{ij} starts at 0 and receives the products $A_{ik}B_{kj}$ in increasing k order: in the blocked kernel the k tiles are visited in increasing order for every (i, j). With no `-ffast-math` and no FMA instructions (no `-march`), the four kernels must therefore return *bitwise identical* results, which the self test and the validation check. The optimisations change only the order in which the n^2 independent sums progress, not the sums themselves.

CSR x CSR -> CSR, numeric pass · c/sparse.c

```
for (int i = 0; i < n; i++) {                        /* pass 1 counted row_ptr already */
    int32_t *ci = C->col + row_ptr[i];  int64_t len = 0;
    for (int64_t p = A->row_ptr[i]; p < A->row_ptr[i+1]; p++) {
        const int k = A->col[p];  const double a = A->val[p];
        for (int64_t q = B->row_ptr[k]; q < B->row_ptr[k+1]; q++) {
            const int j = B->col[q];
            if (mark[j] != i) { mark[j] = i; ci[len++] = j; acc[j] = a * B->val[q]; }
            else acc[j] += a * B->val[q];
        }
    }
    for (int64_t t = 0; t < len; t++) C->val[row_ptr[i] + t] = acc[ci[t]];
}
```

Figure 2: Numeric pass of the row-wise Gustavson product. A preceding symbolic pass with the same loops counts the distinct columns of each row of C, so C is allocated exactly once.

Sparse implementation

`c/sparse.c` (own code, no sparse library) stores matrices in CSR with 64-bit row pointers, 32-bit column indices and binary64 values, i.e. $8(n + 1) + 12z$ bytes for z nonzeros. The product is computed in two passes (symbolic, then numeric) with a dense accumulator `acc` and a marker array, both of length n. Column indices inside a row of C are left in first-touch order (not sorted). Because each C_{ij} again accumulates its products in increasing k order and the skipped dense terms are exact zeros, the CSR result has the same values as the dense kernels. SciPy [12] and NumPy are used only as correctness references.

3.2 Inputs

Part A

The dense inputs are those of Assignment 1: NumPy PCG64 seeded with `[20261012, n]`, values uniform in $[-1, 1)$, binary64 row-major files. The regenerated files are byte-identical to the Assignment 1 files for all 16 shared sizes (SHA-256 in `data/manifest.json`); $n = 3072$ and 4096 were added with the same generator.

Part B

Sparse inputs are written as CSR files read by both programs, so dense and sparse methods see the same logical matrices; the dense program expands them to dense arrays before timing. Each pair A, B has *exactly* $z = \text{round}(dn^2)$ nonzeros, with values uniform in $[-1, 1)$ (exact zeros redrawn), generated with PCG64 seeded with `[20261026, n, round(1000 d%), structure, matrix]`. Density d is the percentage of nonzero entries, and sparsity is $100 - d$. Two nonzero distributions were used:

- **uniform**: z distinct positions drawn uniformly at random from the n^2 positions (rows therefore hold a binomially varying number of nonzeros);
- **banded**: the narrowest band $|i - j| \leq w$ that can hold z entries; the diagonals $|i - j| < w$ are full and the remaining entries are random positions of the two outer diagonals.

3.3 Environment and fixed settings

Table 1 lists the environment; it is the machine of Assignment 1, and `results/environment.json` records the details.

Table 1: Test environment and fixed settings.

CPU	AMD Ryzen 7 7730U (Zen 3), 8 physical cores / 16 logical processors; 32 KB L1d and 512 KB L2 per core, 16 MB L3
Memory, OS	16 GB RAM; Windows 11 Home 10.0.26300; HP Laptop 15-fc0xxx; on AC power, "Balanced" power plan
Compiler	GCC 16.2.0 (MinGW-w64 UCRT); `-O2 -std=c11 -Wall -Wextra` (no `-march`, no fast-math); vectorised loops listed in `results/vectorization_report.txt`
Threads	every kernel is sequential (one thread); no numerical library inside the timed code
Scripts	CPython 3.11.9, NumPy 2.2.5, SciPy 1.17.1, Matplotlib 3.10.1, psutil 7.2.2

3.4 Measurement protocol

Timing boundaries

Dense: allocation and zero-initialisation of C plus the loops; for `transposed` also the allocation and filling of B^T. *CSR multiply:* both passes, the work arrays and the allocation of the output CSR arrays. *Conversions* (Part B, $n \leq 4096$): dense→CSR of A and B together, and CSR→dense of C, each timed in its own loop and reported separately; a "CSR including conversions" figure is their explicit sum. Reading files, expanding CSR inputs for the dense methods, checksums and dumps are outside every timed region. Timer: `QueryPerformanceCounter`.

Repetitions

Each configuration runs in a fresh process: one warm-up run (cache and page-fault effects), excluded from the statistics, then five measured repetitions. Samples shorter than 0.1 s are batched and divided by the batch size. Every result is consumed through a `volatile` sink and a checksum. We report the median and the interquartile range (IQR); all samples, including warm-ups, are in `results/raw/measurements.csv`. Configurations run in order of increasing n with a 2 s pause between processes.

Memory

As in Assignment 1, the driver records the Windows peak working set of the process (`PeakWorkingSetSize`, read with psutil [14]), sampled every 20 ms. We compare it with analytical storage models: $24n^2$ bytes for the dense A, B, C ($32n^2$ for `transposed`) and $8(n + 1) + 12z$ bytes per CSR matrix.

Budgets (declared before the large runs)

300 s per multiplication and 4 GB peak working set per process. A configuration that exceeds a budget is stopped and recorded with its limit, and larger sizes of the same method and input are not attempted. A dense configuration whose storage model ($24n^2$ bytes) already exceeds the memory budget is recorded as over the limit without being launched.

Design of the experiments

- **Pre-study:** tile size $b \in \{16, 32, 64, 128, 256\}$ of the blocked kernel at $n = 512, 1024, 2048$, before the main runs.
- **A, dense:** the four kernels for $n = 64, \ldots, 4096$ (10 sizes).
- **B, matched comparison:** uniform inputs with $n = 1024, 2048$ and $d = 0.1$, 0.5, 1, 2, 5, 10, 25, 50 and 100 %; CSR versus the fastest dense kernel (blocked) at every point, and versus the baseline at $n = 1024$.
- **B, structure:** banded inputs with the same $n = 2048$ and $d = 0.5, 1, 2, 5$ % (CSR only: the dense kernels do the same work for any structure).
- **B, beyond the dense budget:** uniform inputs with $n = 4096$ to 32768 at $d = 0.1$ % and 1 % (CSR), with the blocked kernel attempted at $d = 0.1$ % until it fails.
- **Supplementary:** the dense kernels rebuilt with `-fno-tree-vectorize`, to separate the effect of SIMD from that of the memory-access pattern.

3.5 Correctness checks

Before benchmarking

Both programs run known-answer self tests for every kernel: the same hand-computed 3×3, $AI = A$, $IA = A$, $A \cdot 0 = 0$, non-commuting 2×2 and 1×1 cases as in Assignment 1, plus random matrices of sizes that are not multiples of the tile (7 to 257) for which every optimised kernel must be bitwise identical to the baseline, and sparse inputs with empty rows for which the CSR product must equal the dense baseline (`results/selftest.txt`).

After benchmarking

The result of the last repetition of every configuration is compared with NumPy's `A @ B` on the dense matrices ($n \leq 4096$) or with SciPy's CSR product (larger n), using the element-wise bound $|C_{\text{impl}} - C_{\text{ref}}| \leq 2\gamma_n(|A||B|)$ of Assignment 1. The tolerance is an absolute limit for each element, relative to the corresponding element of $|A||B|$; this avoids rejecting correct results where cancellation makes C_{ij} close to zero. We also check that the nnz of the CSR output equals that of the reference and that all repetitions printed the same checksum.

4 Results

4.1 Correctness

All self tests passed (`results/selftest.txt`). All 97 completed configurations satisfy the tolerance; the largest error was 1.2 % of the bound (`results/validation.csv`). At every size the four dense kernels are *bitwise identical* to each other, and in Part B the CSR result has exactly the same values and the same nnz as the dense kernels on the same input. For $n \geq 8192$ the CSR outputs were compared element-wise with SciPy ($n = 8192$) or, where no dump was kept ($n \geq 16384$), by nnz and checksum (relative difference $\leq 9 \times 10^{-13}$). Every configuration printed the same checksum in all its repetitions. Differences in runtime are therefore due to the order of the operations and the storage format, not to different arithmetic.

4.2 Part A: dense methods

Tile size (pre-study)

Table 2 shows the tile-size sweep. $b = 64$ was fastest at $n = 1024$ and 2048 and within 3.5 % of the best ($b = 128$) at $n = 512$, so it was used for all runs. The sweep shown was repeated with the same protocol, on AC power, after the main campaign; it confirms the value $b = 64$ that had been fixed in the configuration before the main runs. A 64×64 tile of B takes 32 KB, the size of the L1 data cache; one row segment of C (512 B) and of B stay in L1 while the tile is reused from L2.

Table 2: Pre-study: median time of the blocked kernel for each tile size b (5 repetitions; fastest per row in bold). Raw data: `results/tuning/block_sweep.csv`.

n	$b = 16$	$b = 32$	$b = 64$	$b = 128$	$b = 256$
512	38.25 ms	28.25 ms	25.41 ms	**24.56 ms**	37.04 ms
1024	305.32 ms	232.09 ms	**194.19 ms**	240.59 ms	268.84 ms
2048	2.69 s	1.85 s	**1.62 s**	2.03 s	2.24 s

Runtime

Table 3 and Figure 3 give the median runtime of the four kernels. The baseline needed 182 s at $n = 3072$ and did not finish one multiplication within the 300 s budget at $n = 4096$; the three optimised kernels completed every size. Blocking was the fastest method for $n \geq 768$ and took 13.9 s at $n = 4096$.

Table 3: Part A: median time per multiplication and relative IQR (IQR/median, %) over five repetitions. The large IQR of the ikj and blocked kernels at $n = 512$–1024 (up to 35 %) comes from runs shorter than 0.3 s in which two of the five repetitions were 20–45 % slower than the other three (e.g. blocked, $n = 768$: 77–112 ms).

	ijk		ikj		transposed		blocked	
n	Median	IQR%	Median	IQR%	Median	IQR%	Median	IQR%
64	142.8 μs	4.0	56.6 μs	9.6	156.8 μs	7.7	50.7 μs	10.8
128	2.96 ms	4.5	346.8 μs	8.8	1.28 ms	5.9	427.7 μs	4.2
256	34.12 ms	0.9	3.37 ms	7.6	11.48 ms	3.6	3.33 ms	8.5
512	373.56 ms	1.9	27.76 ms	24.4	98.04 ms	2.3	30.22 ms	2.2
768	1.05 s	0.4	96.04 ms	18.0	336.81 ms	1.5	85.75 ms	35.2
1024	5.93 s	0.6	225.96 ms	26.2	799.79 ms	2.2	222.61 ms	1.7
1536	18.24 s	0.4	1.44 s	1.0	2.82 s	0.2	760.86 ms	10.2
2048	49.02 s	0.5	3.42 s	0.6	6.74 s	0.0	1.76 s	3.6
3072	182.04 s	0.5	12.83 s	0.7	22.92 s	0.1	5.72 s	0.7
4096	*timeout*		29.36 s	0.4	54.53 s	0.3	13.89 s	1.7

Figure 3 (right) shows three different behaviours:

- **ijk** cost rises from 0.55 to 6.3 ns per multiply-add, the pattern of Assignment 1. At $n = 1024$ the median, 5.93 s, is 8 % above the 5.48 s of Assignment 1.
- **blocked** stays at 0.19–0.23 ns (about 10 GFLOP/s) from $n = 64$ to 4096: its runtime grows exactly as n^3.
- **ikj** costs the same as blocked up to $n = 1024$, then about twice as much (0.40–0.44 ns) from $n = 1536$ on. **transposed** rises only slowly, from 0.6 to 0.79 ns, but is 1.8–3.7× slower than ikj at every size.

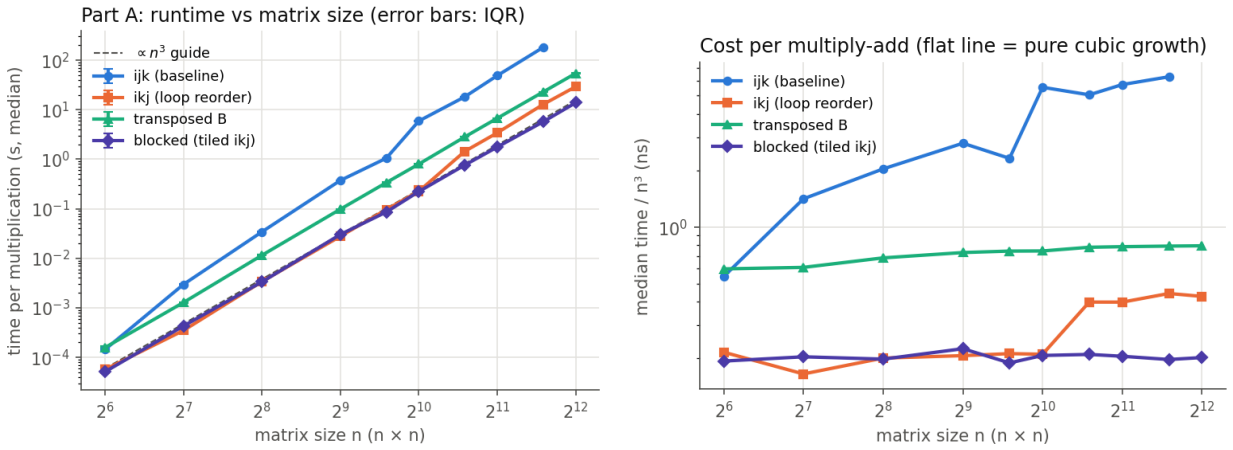


Figure 3: Part A. Left: median time per multiplication (log–log; the IQR bars are smaller than the markers). Right: median time divided by n^3, the cost of one multiply-add; a flat line means pure cubic growth.

Table 4 gives the speed-ups and the achieved GFLOP/s. The speed-up over the baseline grows with n for the blocked kernel, from 2.8× at $n = 64$ to 31.8× at $n = 3072$, and has a local peak at $n = 1024$ (26× for ikj and blocked), where the baseline suffers the conflict misses of power-of-two sizes.

Table 4: Part A: speed-up over the ijk baseline (ratio of medians), throughput $2n^3$/median, and peak working set of the process. The peak working set of ikj and blocked equals that of ijk within 1 MB.

	Speed-up over ijk			GFLOP/s ($2n^3$/median)				Peak WS (MB)	
n	ikj	transp.	blocked	ijk	ikj	transp.	blocked	ijk	transp.
64	2.5	0.9	2.8	3.67	9.27	3.34	10.33	4	4
128	8.5	2.3	6.9	1.42	12.09	3.28	9.81	4	5
256	10.1	3.0	10.3	0.98	9.96	2.92	10.09	6	6
512	13.5	3.8	12.4	0.72	9.67	2.74	8.88	10	12
768	10.9	3.1	12.2	0.86	9.43	2.69	10.56	18	23
1024	26.2	7.4	26.6	0.36	9.50	2.69	9.65	29	38
1536	12.6	6.5	24.0	0.40	5.02	2.57	9.53	61	80
2048	14.3	7.3	27.8	0.35	5.02	2.55	9.74	105	138
3072	14.2	7.9	31.8	0.32	4.52	2.53	10.14	231	306
4096	–	–	–	–	4.68	2.52	9.90	–	541

Memory

The three in-place kernels use the same memory: their peak working set follows the storage model $24n^2$ bytes plus about 4 MB of process baseline (231 MB at $n = 3072$ against 226 MB predicted). The transposed kernel needs one more n^2 array, $32n^2$ bytes (541 MB at $n = 4096$ against 537 MB). Under the 4 GB budget, the storage model limits any of these dense kernels to $n \leq 12\,900$ (in-place) and $n \leq 11\,180$ (transposed); the time budget limits the baseline to $n = 3072$ and the blocked kernel, extrapolating its n^3 growth, to about $n \approx 11\,400$.

Supplementary: vectorisation versus access pattern

Table 5 compares the same source compiled with and without `-fno-tree-vectorize`; the checksums were identical in all runs. SSE2 vectorisation made ikj 1.25–1.65× faster and blocked 1.6–1.7× faster, and had no effect on the transposed kernel, whose inner loop is a reduction that GCC does not vectorise. Without vectorisation the blocked kernel is still 18.5× faster than the baseline at $n = 2048$ (49.0 s / 2.65 s): most of its gain comes from the memory-access pattern, and SIMD adds a factor of about 1.6 on top of it.

Table 5: Supplementary: median time with and without loop vectorisation (same flags otherwise; builds alternated per configuration; 5 repetitions). Raw data: `results/supplementary/vectorization.csv`.

Method	n	-O2 (SSE2 vectorised)	-O2 -fno-tree-vectorize	Ratio
ikj	256	3.38 ms	5.56 ms	1.65×
ikj	512	27.91 ms	44.43 ms	1.59×
ikj	1024	256.26 ms	329.66 ms	1.29×
ikj	2048	3.31 s	4.14 s	1.25×
blocked	256	3.09 ms	5.29 ms	1.71×
blocked	512	27.67 ms	44.05 ms	1.59×
blocked	1024	192.85 ms	322.15 ms	1.67×
blocked	2048	1.64 s	2.65 s	1.62×
transposed	256	11.59 ms	11.60 ms	1.00×
transposed	512	96.01 ms	99.95 ms	1.04×
transposed	1024	795.54 ms	794.73 ms	1.00×
transposed	2048	6.70 s	6.71 s	1.00×

4.3 Part B: sparse versus dense

Matched comparison

Table 6 and Figure 4 compare the CSR product with the blocked kernel, and at $n = 1024$ also with the baseline, on the same uniform inputs. The dense kernels do the same work for any density, so their time is flat. The CSR time grows with density, from 8.8 µs to 1.51 s at $n = 1024$. CSR was faster than the blocked kernel at every density up to 25 % and slower at 50 % and 100 %, at both sizes; with the conversions included the crossover stays between 25 % and 50 %.

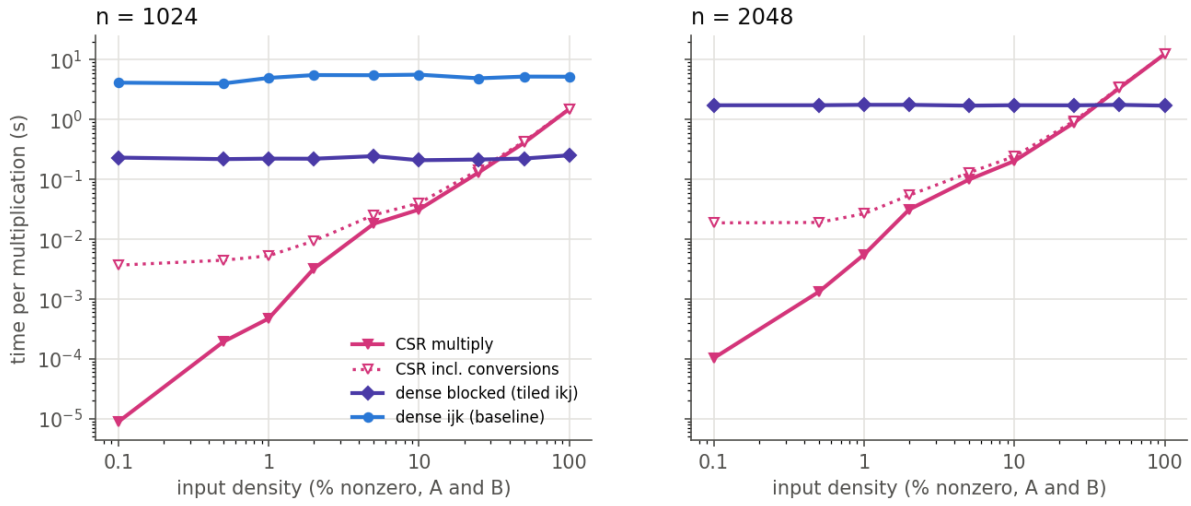


Figure 4: Part B: median time per multiplication against input density, for the same logical inputs. The dotted line adds the separately measured conversion times to the CSR multiplication.

Table 6: Part B, uniform inputs: nonzeros, output density, median CSR multiplication time, CSR time including the conversions (dense→CSR of A and B plus CSR→dense of C, summed explicitly), and the dense kernels on the same logical inputs. †Speed-up = blocked / CSR multiplication (both medians). nnz(B) equals nnz(A) by construction.

n	Dens. %	nnz(A)	nnz(C)	Out dens. %	CSR mult.	+ conversions	Blocked	ijk	Speed-up†
1024	0.1	1 049	1 123	0.107	8.8 µs	3.68 ms	231.57 ms	4.15 s	26 250×
1024	0.5	5 243	26 628	2.54	192.6 µs	4.45 ms	218.31 ms	4.02 s	1 134×
1024	1	10 486	101 969	9.72	471.6 µs	5.27 ms	222.57 ms	4.98 s	472×
1024	2	20 972	352 086	33.6	3.23 ms	9.38 ms	221.75 ms	5.56 s	68.7×
1024	5	52 429	967 449	92.3	18.13 ms	25.45 ms	244.92 ms	5.53 s	13.5×
1024	10	104 858	1 048 540	100	31.31 ms	40.11 ms	209.29 ms	5.64 s	6.69×
1024	25	262 144	1 048 576	100	131.29 ms	144.95 ms	215.05 ms	4.91 s	1.64×
1024	50	524 288	1 048 576	100	410.17 ms	432.79 ms	223.28 ms	5.25 s	0.544×
1024	100	1 048 576	1 048 576	100	1.51 s	1.52 s	253.11 ms	5.22 s	0.168×
2048	0.1	4 194	8 673	0.207	102.3 µs	18.73 ms	1.74 s	–	16 985×
2048	0.5	20 972	209 270	4.99	1.31 ms	19.06 ms	1.74 s	–	1 324×
2048	1	41 943	776 382	18.5	5.52 ms	26.66 ms	1.77 s	–	320×
2048	2	83 886	2 346 584	55.9	31.48 ms	54.42 ms	1.77 s	–	56.1×
2048	5	209 715	4 169 706	99.4	99.55 ms	129.83 ms	1.71 s	–	17.2×
2048	10	419 430	4 194 304	100	203.18 ms	242.80 ms	1.74 s	–	8.57×
2048	25	1 048 576	4 194 304	100	889.11 ms	950.11 ms	1.73 s	–	1.95×
2048	50	2 097 152	4 194 304	100	3.38 s	3.47 s	1.77 s	–	0.522×
2048	100	4 194 304	4 194 304	100	12.48 s	12.53 s	1.71 s	–	0.137×

Conversion costs

Table 7 separates the costs. Converting A and B to CSR scans $2n^2$ dense entries, so it costs at least 2.7 ms at $n = 1024$ and grows with the number of nonzeros written (20 ms at 50 %); converting C back costs 1–2.6 ms. At 0.1–0.5 % density the conversions take 13–420× longer than the multiplication itself, yet CSR including conversions is still 63× ($n = 1024$) and 93× ($n = 2048$) faster than the blocked kernel at 0.1 %. From 2–5 % on the multiplication dominates.

Output fill-in

The output is much denser than the inputs (Figure 5). At $n = 2048$, inputs with 2 % nonzeros give a product with 55.9 % nonzeros, and from 5 % on C is practically dense. The measured output densities follow the model $1 - (1 - d^2)^n$ closely, and they also depend on n: the same 1 % input density gives 9.7 % output density at $n = 1024$ and 18.5 % at $n = 2048$.

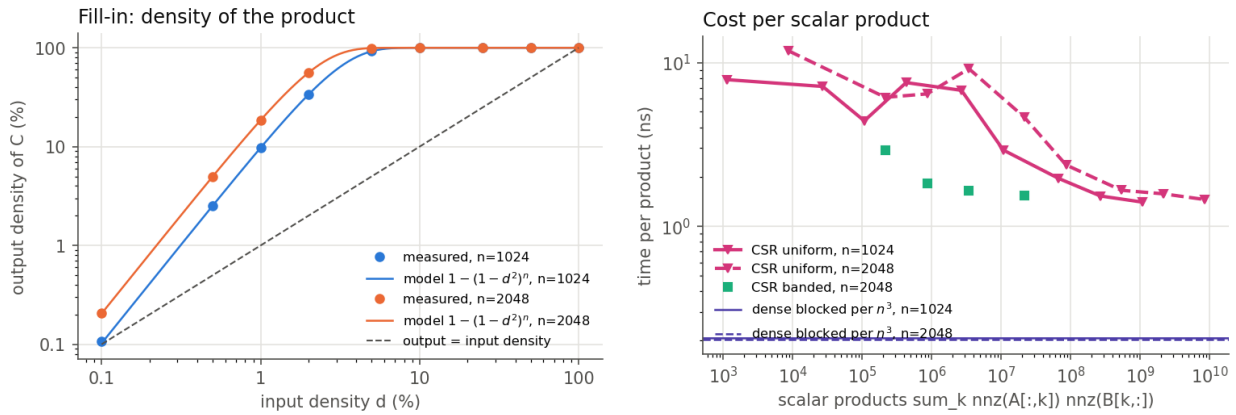


Figure 5: Left: density of the product against the input density, with the model for independent uniform inputs. Right: CSR time per scalar product against the number of products W; the horizontal lines are the blocked kernel's time per multiply-add (median/n^3).

Table 7: Part B, uniform inputs: CSR cost breakdown (medians), time per scalar product (multiply time/W, with $W = \sum_k \mathrm{nnz}(A_{:,k})\,\mathrm{nnz}(B_{k,:})$), peak working set, and the analytical CSR storage of A, B and C. *Process of the main run, which after the multiplications also expands A and B to dense arrays to time the conversions; "CSR only" is a supplementary run of the multiplication alone (`results/supplementary/sparse_memory.csv`).

		CSR time (median)				Peak WS (MB)			Model (MB)
n	Dens. %	A,B→CSR	Multiply	C→dense	ns/product	CSR*	CSR only	Blocked	CSR A, B, C
1024	0.1	2.66 ms	8.8 μs	1.01 ms	7.86	21	4.1	29	0.1
1024	0.5	2.84 ms	192.6 μs	1.42 ms	7.14	22	4.5	29	0.5
1024	1	3.22 ms	471.6 μs	1.58 ms	4.40	23	5.6	29	1.5
1024	2	4.05 ms	3.23 ms	2.10 ms	7.53	26	8.8	29	4.8
1024	5	4.82 ms	18.13 ms	2.50 ms	6.76	35	17.0	30	12.9
1024	10	6.26 ms	31.31 ms	2.54 ms	2.92	38	19.2	30	15.1
1024	25	11.28 ms	131.29 ms	2.39 ms	1.96	46	23.0	29	18.9
1024	50	20.32 ms	410.17 ms	2.30 ms	1.53	59	29.3	29	25.2
1024	100	10.09 ms	1.51 s	2.59 ms	1.40	84	41.8	33	37.8
2048	0.1	11.67 ms	102.3 μs	6.96 ms	11.79	72	4.4	105	0.3
2048	0.5	11.53 ms	1.31 ms	6.21 ms	6.12	75	8.0	105	3.1
2048	1	13.18 ms	5.52 ms	7.95 ms	6.43	83	14.6	105	10.4
2048	2	13.20 ms	31.48 ms	9.74 ms	9.16	104	34.4	106	30.2
2048	5	18.72 ms	99.55 ms	11.55 ms	4.64	131	59.2	106	55.1
2048	10	24.60 ms	203.18 ms	15.02 ms	2.37	142	64.5	105	60.4
2048	25	47.05 ms	889.11 ms	13.95 ms	1.66	172	79.6	105	75.5
2048	50	79.66 ms	3.38 s	10.89 ms	1.58	222	104.8	105	100.7
2048	100	45.28 ms	12.48 s	9.99 ms	1.45	323	155.1	121	151.0

Cost per scalar product

The CSR time per scalar product (Figure 5, right) is 4–12 ns while the products per row are few (density $\leq 5\,\%$) and falls to 1.4–1.7 ns at high density, where each row of C receives many products into an accumulator that stays in cache. Even at its best, a CSR product costs about 7× more than a multiply-add of the blocked kernel (0.21 ns), because each one needs an index load, an indirect access to the accumulator and a marker test, and it is not vectorised.

Memory

Figure 6 shows the peak working set. The CSR multiplication alone follows the storage model of A, B and C plus the 4 MB process baseline; because C fills in, its memory grows with density much faster than that of the inputs. The CSR multiplication needed less memory than the blocked kernel up to 25 % density at both sizes (23 against 29 MB at $n = 1024$), about the same at 50 %, and more at 100 % density, where CSR stores 12 bytes per entry against 8.

Structure

Table 8 compares banded and uniform inputs with the same $n = 2048$, density and number of nonzeros. Their numbers of scalar products differ by less than 0.5 %, but the banded products are 5–14× sparser, because a product of two bands is again a band, and the CSR multiplication is 2.1–5.6× faster (1.5–2.9 ns per product instead of 4.6–9.2 ns), because the rows of B that are combined touch neighbouring columns of the accumulator.

Largest feasible sizes

Table 9 and Figure 7 extend the comparison beyond the dense budget. At 0.1 % density the blocked kernel took 112 s at $n = 8192$ and was not attempted at $n = 16384$: its storage model ($24n^2 = 6.4$ GB) exceeds the 4 GB budget. CSR multiplied the same $n = 8192$ matrices in 4.6 ms and reached $n = 32768$ in 0.30 s with 447 MB, a size at which the three dense matrices would need 25.8 GB. At 1 % density the output fills in (80.6 % at $n = 16384$, 216 million nonzeros, 2.7 GB)

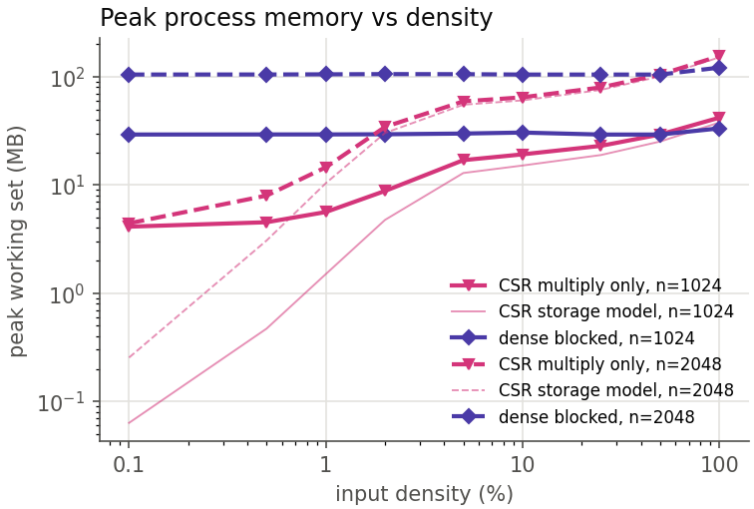


Figure 6: Peak working set of the CSR multiplication alone (supplementary run without conversions) and of the blocked kernel on the same inputs, with the CSR storage model of A, B and C.

Table 8: Part B, $n = 2048$: uniform versus banded inputs with the same density and nnz. "Products" is $W = \sum_k \mathrm{nnz}(A_{:,k})\,\mathrm{nnz}(B_{k,:})$.

Dens. %	Structure	nnz(A)	Products	nnz(C)	CSR mult.	ns/product
0.5	uniform	20 972	214 716	209 270	1.31 ms	6.12
0.5	banded	20 972	214 859	39 857	624.7 µs	2.91
1	uniform	41 943	858 880	776 382	5.52 ms	6.43
1	banded	41 943	859 722	81 651	1.57 ms	1.83
2	uniform	83 886	3 435 637	2 346 584	31.48 ms	9.16
2	banded	83 886	3 441 698	164 921	5.67 ms	1.65
5	uniform	209 715	21 473 845	4 169 706	99.55 ms	4.64
5	banded	209 715	21 564 409	412 091	33.02 ms	1.53

and the CSR run at $n = 32768$ was stopped when it exceeded the 4 GB budget; the fill-in model predicts about 10^9 output nonzeros (12 GB).

Table 9: Part B beyond the dense budget (uniform inputs). The CSR peak working set at $n = 4096$ includes the dense buffers of the conversion timing (conversions were timed only for $n \le 4096$). "memory limit" at blocked $n = 16384$: not attempted, storage model above 4 GB.

n	Dens. %	nnz(A)	nnz(C)	Out dens. %	CSR mult.	CSR peak WS	Blocked dense
4096	0.1	16 777	68 484	0.408	595.5 µs	274 MB	13.70 s
8192	0.1	67 109	547 629	0.816	4.61 ms	13 MB	112.18 s
16384	0.1	268 435	4 359 773	1.62	32.11 ms	63 MB	*memory limit*
32768	0.1	1 073 742	34 618 458	3.22	296.01 ms	447 MB	–
4096	1	167 772	5 639 389	33.6	54.68 ms	348 MB	–
8192	1	671 089	37 526 496	55.9	534.97 ms	471 MB	–
16384	1	2 684 355	216 285 189	80.6	4.09 s	2664 MB	–
32768	1	–	–	–	*memory limit*	4001 MB	–

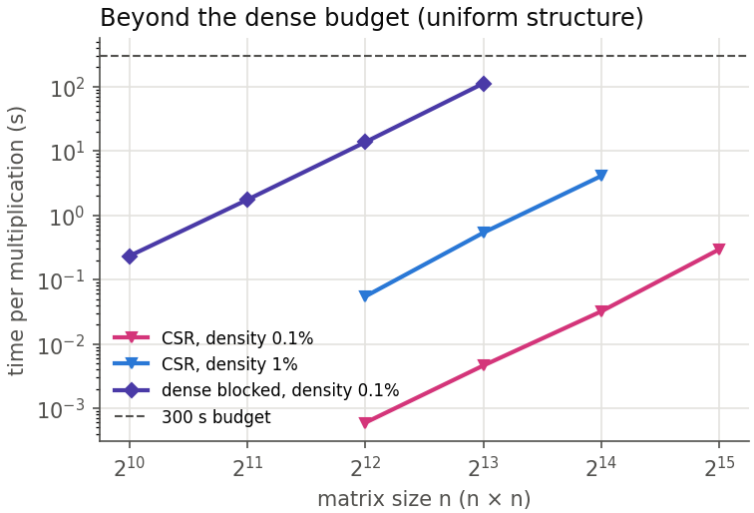


Figure 7: Median time per multiplication of CSR (0.1 % and 1 % density) and of the blocked kernel (0.1 %) up to the largest size that fits the budgets.

Limits reached

Table 10 lists every configuration that hit a budget (`results/summary/limits.csv`).

Table 10: Largest completed size per method and the limit that stopped it.

Method	Input	Largest n	Limit reached
ijk (baseline)	dense	3072	$n = 4096$: no multiplication within 300 s
ikj, transposed, blocked	dense	4096	none (largest tested size)
blocked	uniform 0.1 %	8192	$n = 16384$: storage model 6.4 GB > 4 GB, not attempted
CSR	uniform 0.1 %	32768	none (largest tested size)
CSR	uniform 1 %	16384	$n = 32768$: peak working set > 4 GB (output fill-in)

Deviations from the plan

Two events interrupted the campaign. (i) During the first run of ijk at $n = 3072$ the laptop entered Modern Standby for seven minutes (Windows Kernel-Power events 506/507); that run and the following $n = 4096$ run were discarded (`results/raw/excluded.csv`), a keep-awake request was added, and both configurations were rerun. (ii) The benchmarking session was stopped because the laptop ran low on system memory while the driver attempted the blocked kernel at $n = 16384$ (6.4 GB of dense storage, with other applications open); that attempt was not recorded, the driver was then given a check of the dense storage model against the budget before launching a dense run, and the remaining configurations were run with a `-resume` option that skips completed ones. Each configuration runs in its own process, so completed results were not affected.

5 Discussion

Part A: what makes the optimised kernels faster

All four kernels do the same arithmetic in the same order for each C_{ij}, and the results are bitwise identical, so the differences in Table 3 come only from how the operations are scheduled. The baseline reads a column of B in its inner loop; as in Assignment 1, its cost per multiply-add grows tenfold with n. Reordering the loops to i-k-j makes the inner loop stream a row of B and a row of C with unit stride and independent iterations, which removes the strided access and lets GCC use SSE2 vectors. This is enough to run at about 10 GFLOP/s while B (8 MB at $n = 1024$) fits in

the 16 MB L3 cache. From $n = 1536$ ($B = 18.9$ MB) every row i streams all of B from DRAM again, and the cost per multiply-add doubles. Blocking restricts the inner loops to a 64×64 tile of B that is reused for 64 rows of C, so the traffic to memory falls by a factor of about b and the cost per multiply-add stays flat up to $n = 4096$. H1 is therefore only partly supported: the constant cost per multiply-add holds for the blocked kernel, and for ikj only while B fits in L3; the speed-up over the baseline did grow with n and was higher at the power-of-two sizes than at their neighbours, but clearly so only at $n = 1024$ (26× for both kernels, against 11–24× at $n = 768$ and 1536). Elsewhere the effect was smaller (ikj at $n = 512$: 13.5× against 10.9× at $n = 768$) or absent (blocked at $n = 512$ and 2048). H2 is supported: blocking was the fastest kernel from $n = 768$, and ikj was as fast as blocking for $n \leq 1024$.

The transposed kernel removes the strided access too, but stays at about 0.8 ns per multiply-add. Its inner loop is a reduction into one scalar, so each addition must wait for the previous one, and GCC may not vectorise it without reassociation (Table 5). It is limited by the latency of the floating-point additions, not by memory. A loop order that allows independent updates was therefore more important than a contiguous layout alone. The cost of the transposition itself is small: one n^2 copy against n^3 multiply-adds.

The supplementary experiment separates two effects that a single speed-up figure mixes. Of the 27.8× of blocking at $n = 2048$, a factor of about 18.5 remains without SIMD, so the access pattern accounts for most of the gain. The 1.6× from vectorisation is below the factor of two that 16-byte vectors would allow, which also suggests that memory, not arithmetic, limits these kernels. A library such as OpenBLAS would add register blocking, 32-byte AVX2 FMA vectors and multiple threads [4]; these were outside the scope of this sequential study.

Part B: optimisation effect versus sparse-storage effect

A speed-up of CSR over the ijk baseline mixes two effects. At $n = 1024$ and 1 % density the CSR product was 10 600× faster than the baseline; of this, a factor of 22 is the dense optimisation (ijk → blocked, on the same input) and a factor of 472 is the sparse storage (blocked → CSR). Even at 100 % density, CSR was 3.5× faster than the baseline because Gustavson's algorithm reads the rows of B in i-k-j order, while it was 6× slower than the blocked kernel. All Part B speed-ups are therefore reported against the blocked kernel, the fastest dense method.

When the sparse representation pays off

Against the blocked kernel, CSR saves the work on zeros (its work is $W \approx n^3 d^2$ instead of n^3) but pays about 1.4–12 ns per product instead of 0.21 ns. With the measured high-density cost of 1.5 ns per product, the break-even density is $d \approx \sqrt{0.21/1.5} \approx 37\,\%$, which agrees with the measured crossover between 25 % and 50 % at both sizes. H3 is supported at low density but not for the crossover: it was not "of the order of ten percent" but between 25 % and 50 %, because the cost per product fell at high density. For memory, the crossover is similar (25–50 %) because the output fills in: at these densities C is dense, and CSR needs 12 bytes per entry.

Density alone does not determine performance

Three results show that the percentage of zeros is not enough to predict the outcome (H4, supported). First, the output density depends on n as well as on the input density (Figure 5), and the output size drives both the memory and the cost of writing C. Second, the CSR cost per product varied by a factor of 8 between densities, so the time is not proportional to the number of nonzeros. Third, at the same density, nnz and number of products, banded inputs produced 5–14× fewer output nonzeros and ran up to 5.6× faster than uniform inputs. What determines the CSR runtime is the number of scalar products, the number of distinct output entries and the locality of the accumulator accesses, all of which depend on the structure.

Conversions and scope

Conversions matter only when the data are produced in dense form: at low density, converting A and B cost up to 420 times more than the multiplication, but the total was still far below the dense multiplication time because a conversion is $O(n^2)$ and the dense product $O(n^3)$. In applications where the matrices are created in sparse form, the conversion disappears, and the largest feasible size is limited by the output fill-in instead of n^2 storage: CSR handled $n = 32768$ at 0.1 % density but not at 1 %.

Limitations

- One machine, one compiler and one set of flags; no `-march`, so no AVX2 or FMA. The relative ranking of the dense kernels may change with wider vectors or other cache sizes.
- The baseline varied between processes: on the Part B inputs of $n = 1024$ (same kernel, same work) its median ranged from 4.0 to 5.6 s, and from 5.48 s (Assignment 1) to 5.93 s on the dense inputs. Assignment 1 found this kernel sensitive to memory layout; we did not investigate the cause here. Speed-ups relative to the baseline carry this uncertainty (±15 %), those of the blocked kernel do not (209–253 ms).
- Synthetic inputs: uniform and banded structures only. Real sparse matrices (power-law rows, blocks) would place the crossover elsewhere; the CSR output columns are unsorted, which some consumers would have to sort at extra cost.
- The memory metric is the peak working set of the whole process, sampled every 20 ms. It includes the process baseline (about 4 MB) and, in the main Part B runs, the conversion buffers; the supplementary run isolates the multiplication.
- No hardware counters were used: the explanations of the L3 and latency limits are consistent with the measurements but not confirmed by cache-miss counts.

6 Conclusions

Key findings

On this machine and with these settings:

- All optimised dense kernels gave **bitwise-identical results** to the baseline, so their differences are purely in how they access memory and use the processor.
- **Cache blocking** was the fastest dense method: about 10 GFLOP/s at every size, 27.8× faster than the baseline at $n = 2048$ and 31.8× at $n = 3072$; the baseline could not finish $n = 4096$ in 300 s, blocking took 13.9 s.
- **Loop reordering** alone was as fast as blocking while B fit in the L3 cache and half as fast beyond it; the **transposed** layout was limited by the latency of its scalar reduction. Vectorisation contributed a factor of 1.25–1.7, the access pattern the rest.
- **CSR** beat the fastest dense kernel by more than two orders of magnitude at densities ≤ 1 % and by 1.6–2× at 25 %, and lost at 50 % and above; the conversions dominated the CSR time up to 1–2 % density but never removed its advantage.
- The **sparse advantage was limited by fill-in and structure**, not by the input density alone: products of sparse matrices became dense quickly, and inputs with the same density but a banded structure ran up to 5.6× faster.
- CSR extended the largest feasible size from $n = 8192$ (dense, time and memory budgets) to $n = 32768$ at 0.1 % density; at 1 % the output fill-in stopped it at $n = 16384$.

Open questions

Whether register blocking and AVX2/FMA (`-march=native`) change the ranking; a hardware-counter confirmation of the cache explanations; and how the crossover moves for real sparse matrices with irregular structure.

Reproducibility materials

Repository: https://github.com/amadoartiiles/matrix-multiplication-optimised-sparse. It contains:

- source code: `c/common.h`, `c/dense.c` (baseline kernel body copied unchanged from Assignment 1, https://github.com/amadoartiiles/matrix-multiplication-studies), `c/sparse.c`;
- configuration: `config/benchmark_config.json` (sizes, densities, seeds, repetitions, budgets, flags, tile size);
- input generation, `scripts/generate_inputs.py` (SHA-256 of every input in `data/manifest.json`), and the build, tuning, benchmark, validation, supplementary and analysis scripts;
- all raw measurements: `results/raw/` (with the excluded rows and the reason for excluding them), `results/tuning/` and `results/supplementary/`, plus the summaries and figures;
- a README with exact build and run commands and dependency versions.

Use of generative AI

Claude (Anthropic) was used to help write the implementations, the benchmarking and analysis scripts, and a draft of this report. All numbers in this report come from running the code on the machine described above; the raw data are included. The author reviewed the code, the results and the text.

References

[1] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins Univ. Press, 2013.

[2] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., 2007. [Online]. Available: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

[3] M. S. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. ASPLOS-IV*, 1991, pp. 63–74.

[4] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Trans. Math. Softw.*, vol. 34, no. 3, art. 12, 2008.

[5] V. Strassen, "Gaussian elimination is not optimal," *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.

[6] GNU Project, "GCC 12 Release Series: Changes, New Features, and Fixes," 2022. [Online]. Available: https://gcc.gnu.org/gcc-12/changes.html

[7] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA, USA: SIAM, 2003.

[8] F. G. Gustavson, "Two fast algorithms for sparse matrices: Multiplication and permuted transposition," *ACM Trans. Math. Softw.*, vol. 4, no. 3, pp. 250–269, 1978.

[9] R. E. Bank and C. C. Douglas, "Sparse matrix multiplication package (SMMP)," *Advances in Computational Mathematics*, vol. 1, pp. 127–137, 1993.

[10] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA, USA: SIAM, 2006.

[11] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[12] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[13] Advanced Micro Devices, "Software Optimization Guide for AMD Family 19h Processors," Pub. no. 56665, 2020.

[14] G. Rodola, "psutil documentation," 2026. [Online]. Available: https://psutil.readthedocs.io/